Lesson 4 Teacher's Key

Unreal

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Requires Setup's collision changes actually done on both objects: Collision Presets = OverlapAllDynamic on BP_Enemy's collision component AND on BP_Bullet's (not just one of the two), plus Sweep checked on BP_Bullet's Add Actor World Offset node from Lesson 3 - watch for students who only touch BP_Enemy and leave BP_Bullet on its Lesson 3 default (BlockAllDynamic, no Sweep), which silently prevents the overlap from ever registering no matter how correct the graph below is. BP_Enemy's own Event Graph handles detection and cleanup entirely once that's done - BP_Bullet needs no new nodes at all:

```
BP_Enemy's Event Graph:
Event ActorBeginOverlap(OtherActor) ->
  Cast To BP_Bullet(OtherActor) ->
    Success: Destroy Actor(target = Cast result)
             Destroy Actor(target = Self)
```

Level 2 - Complete Worked Solution

Destroy Actor is safe to call more than once on the same Actor - a repeat call on an Actor that's already pending destruction is simply ignored - so even the rare case of two bullets overlapping the enemy on the same tick doesn't cause an error. No extra nodes are needed for Level 2.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: play on hit

Add a Play Sound 2D node (using an imported sfx_hit.wav Sound Wave asset) right after the successful Cast:

```
Success: Play Sound 2D(sfx_hit.wav)
         Destroy Actor(target = Cast result)
         Destroy Actor(target = Self)
```

Visual: give the enemy its own look

Create a dynamic material instance and set its Color parameter (the one M_Enemy_Red's Setup step built specifically so this would work) in BP_Enemy's own BeginPlay - since that parameter multiplies against the enemy's real sprite instead of replacing it outright, Dark Red genuinely darkens the red invader texture instead of doing nothing:

```
BP_Enemy's Event Graph:
BeginPlay -> Create Dynamic Material Instance(Self's Material) -> Set
Vector Parameter Value("Color", Dark Red)
```

Structure: keep hit-response inside BP_Enemy

Keep everything about how the enemy responds to being hit - the Cast, both Destroy Actor nodes, the sound - inside BP_Enemy's own Event Graph. BP_Bullet doesn't need any new nodes at all this week; it only needed to exist as something BP_Enemy's overlap event could detect.

Complete Level 3 solution (all 3 goals combined)

Everything above combined, entirely inside BP_Enemy's own Event Graph:

```
BP_Enemy's Event Graph:
BeginPlay -> Create Dynamic Material Instance(Self's Material) -> Set
Vector Parameter Value("Color", Dark Red)
Event ActorBeginOverlap(OtherActor) ->
  Cast To BP_Bullet(OtherActor) ->
    Success: Play Sound 2D(sfx_hit.wav)
             Destroy Actor(target = Cast result)
             Destroy Actor(target = Self)
```

Alternate Path Notes: C++

OnActorBeginOverlap.AddDynamic() bound in BeginPlay() is the templated-delegate equivalent of wiring the Event ActorBeginOverlap node in Blueprint - same underlying engine event, just subscribed to from code.

The Level 3 goals map onto C++ calls the same way they map onto Blueprint nodes: UGameplayStatics::PlaySound2D() for sound, and a UMaterialInstanceDynamic's SetVectorParameterValue() for the visual idea, both called from ACppEnemy's own BeginPlay()/OnOverlap() - keeping the same Structure principle as the Blueprint path.

- Showing both C++ and Blueprint side by side for the same behavior is the clearest way to demystify Blueprints for students who assume "visual" means "not real code" - it's the same engine calls either way.
- This is squarely the optional advanced track - most students should stay on Blueprints; C++ is for students already comfortable with typed languages from Godot, GameMaker, or Unity's C# path.